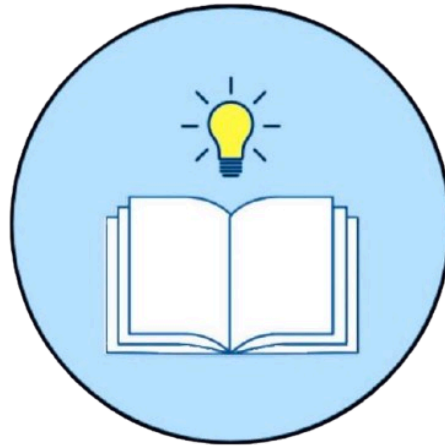


COMTUTOR

Developer Guide



Author(s):

Dahlin Wiebe, wiebed28@mymacewan.ca

James Dwyer, dwyerj5@mymacewan.ca

Zhire Pelayo, pelayoz@mymacewan.ca

Jim Clelland, clellandj3@mymacewan.ca

Kamar El-hayouni, elhayounik@mymacewan.ca

Table of Contents

1	Getting Started	3
	Prerequisites	
	Third Party APIs	
2	Implemented User Stories	4
3	System Modeling	5
	Structural	
	Behavioural	12
4	Appendix of Code	16
5	Testing Strategies	16
	Bug Tracking	22

Getting Started

Prerequisites

Android Studio: Minimum - LadyBug Feature Drop | 2024.2.2 Patch 2, Target - Latest Stable

Kotlin: v2.0.x (updates with Android studio)

Android SDK: Android 15.0 ("VanillaIceCream")

Android SDK Build Tools: Min - 26, Target - 35

Third Party APIs

Firebase Firestore - no-SQL document based database

Firebase Authentication - login authentication server

Firebase Analytics - statistics and logging

Cloudinary Storage - Documentation uploading and storage

Android Room - On-device SQL storage

Duplicating Environment

Clone project from github

```
git clone https://github.com/macewanCS/W25_B1_Repo.git
```

Add SHA-1 and SHA-256 fingerprints to firebase

- Keys can be found by running "gradle signingreport" in the execute gradle function of the gradle task tab
- These keys can be added to the project from the project settings of the Firebase console under "SHA certificate fingerprints"

Sync gradle files

Implemented User Stories

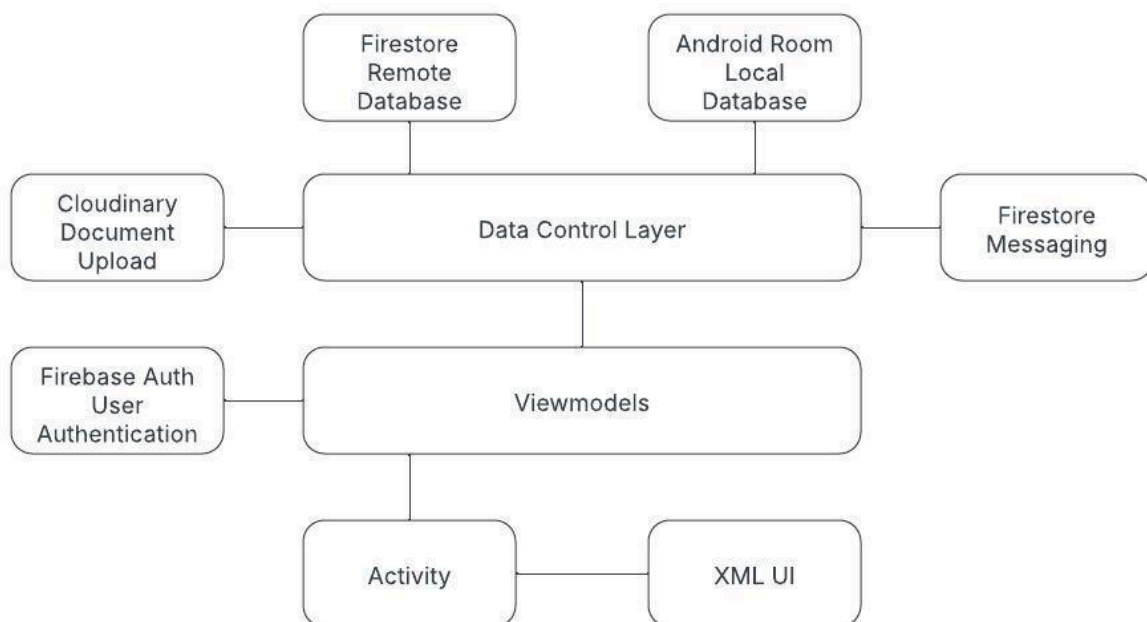
- As a student, I want to be able to sign in so it will remember my information.
- As a student, I want to be able to sign out since other people may be using the device.
- As a student, I want to pick the subject that I am learning in the app so that I can be matched with tutors teaching that subject.
- As a student, I want to specify the grade I am in on the app so that the tutors qualified to teach that level are a match with me.
- As a student, I want to be able to choose tags to categorize the tutor based on their personality and qualifications to narrow down the tutor I am looking for to teach me my subject.
- As a tutor, I want to keep track of each lesson booked through a visual schedule so I can stay organized.
- As a tutor, I want to be able to sign in to the app with a tutor status so that I can have tutor capabilities within the app.
- As a user, I want a dropdown menu to navigate through the app.
- As a student, I want to be able to cancel my session based on my availability.
- As a student, I want to be able to download learning materials for offline use
- As a student, I want to be able to make last-minute bookings on the app so I can freely change my scheduling based on my situation.
- As a student, I want to link my email to receive confirmation of bookings.
- As a student, I want a chat window to reach out to a tutor within the app so that they may answer questions I may have outside a session.
- As a tutor, I want to be able to submit my weekly availability in advance so that I can be assigned students who fit my schedule.
- As a tutor, I want to be able to upload learning materials so that students have additional resources they can use.

- As a tutor, I want to get an email notification if a session was cancelled so that I can stay updated with schedule changes
- As a tutor, I want an email notification if a student has reached out to me on the app
- As a tutor, I want to be able to showcase my areas of expertise, so that I can be matched with a student that I am qualified to teach.
- As a tutor I want an email notification if I was approved or denied.
- As an admin I want to be able to view certificates and approve a new tutor

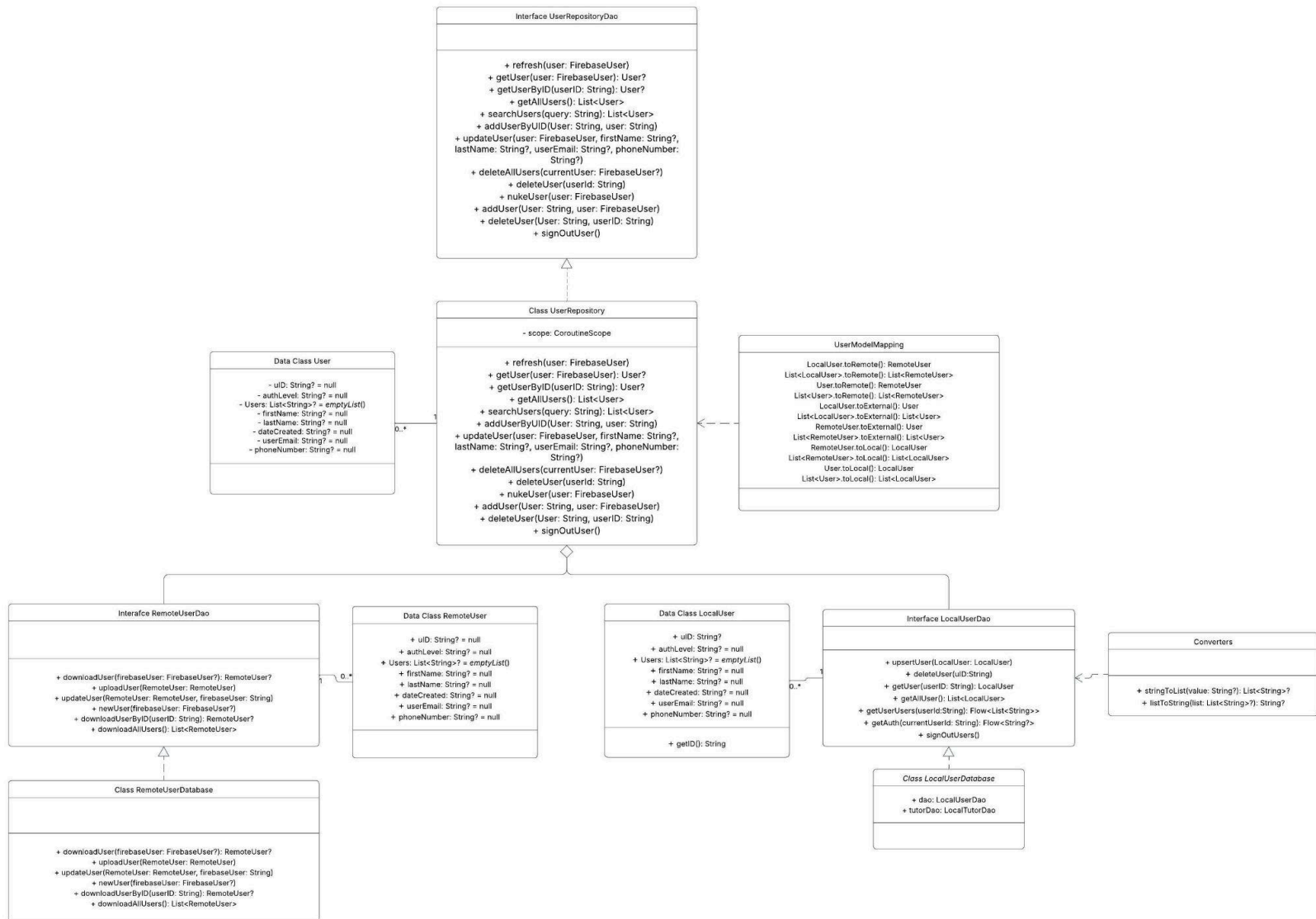
System Modeling

System Structure

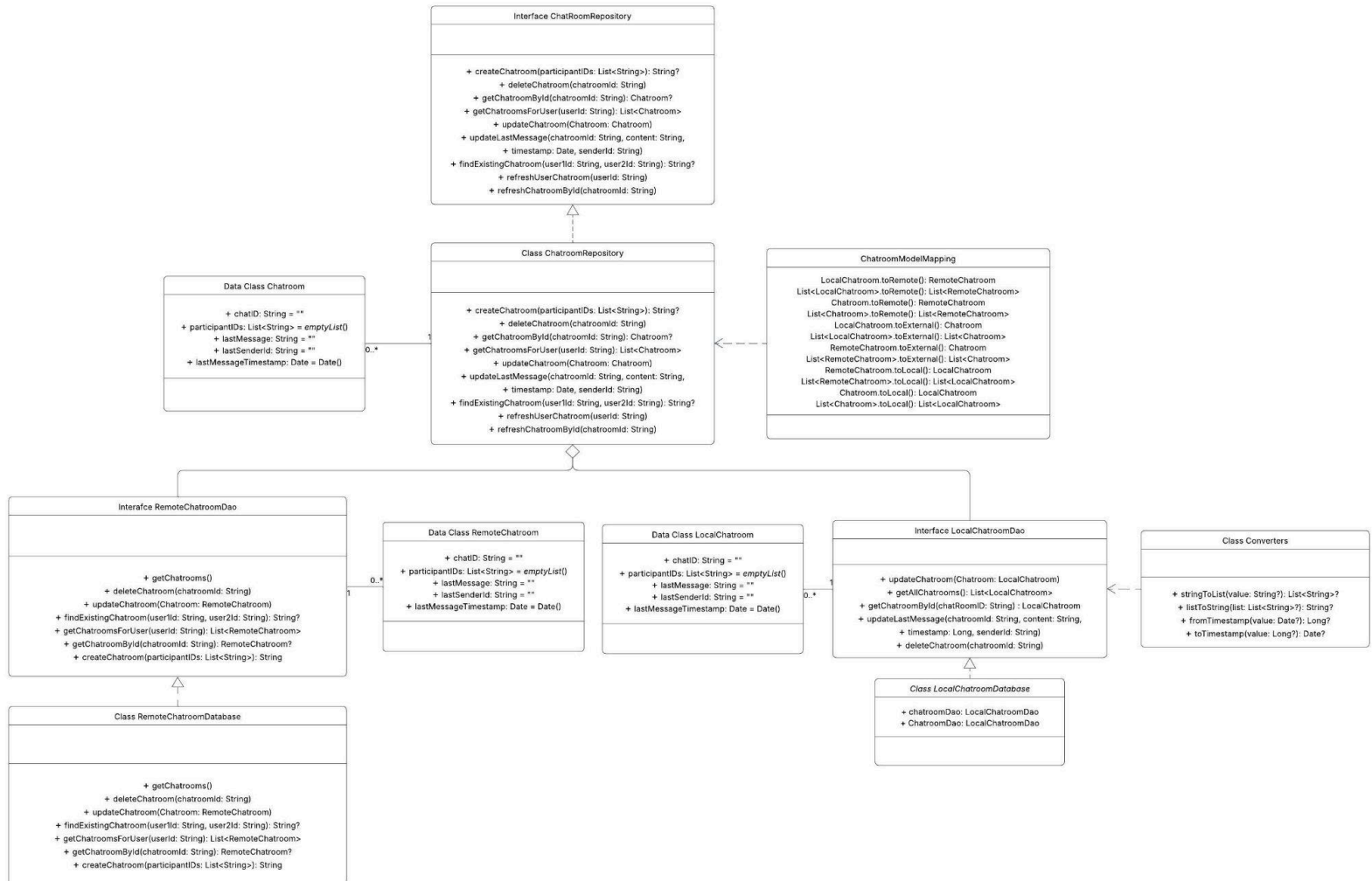
General Architecture Overview



Detailed User Data Repository Class Diagram



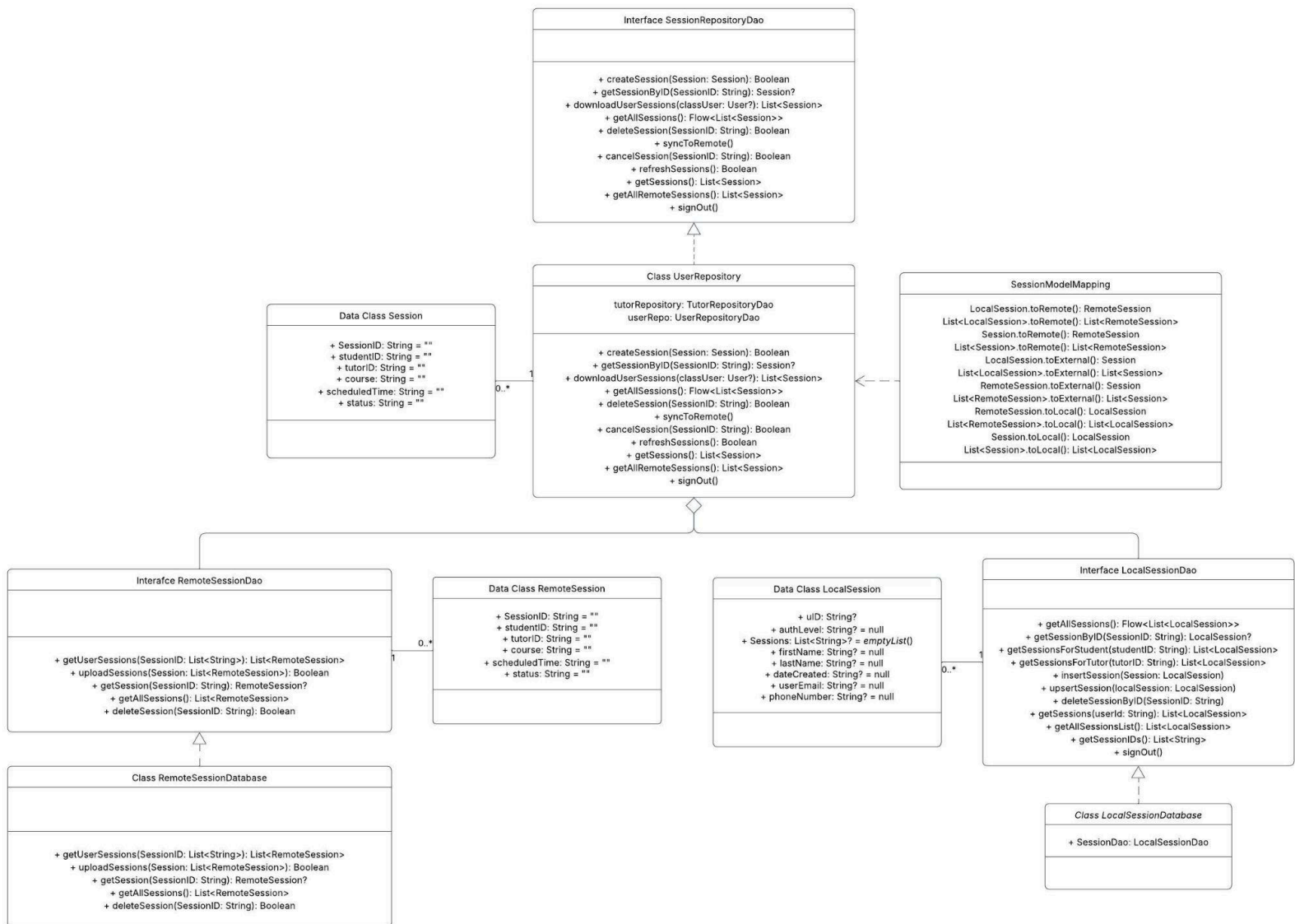
Detailed Chatroom Class Diagram



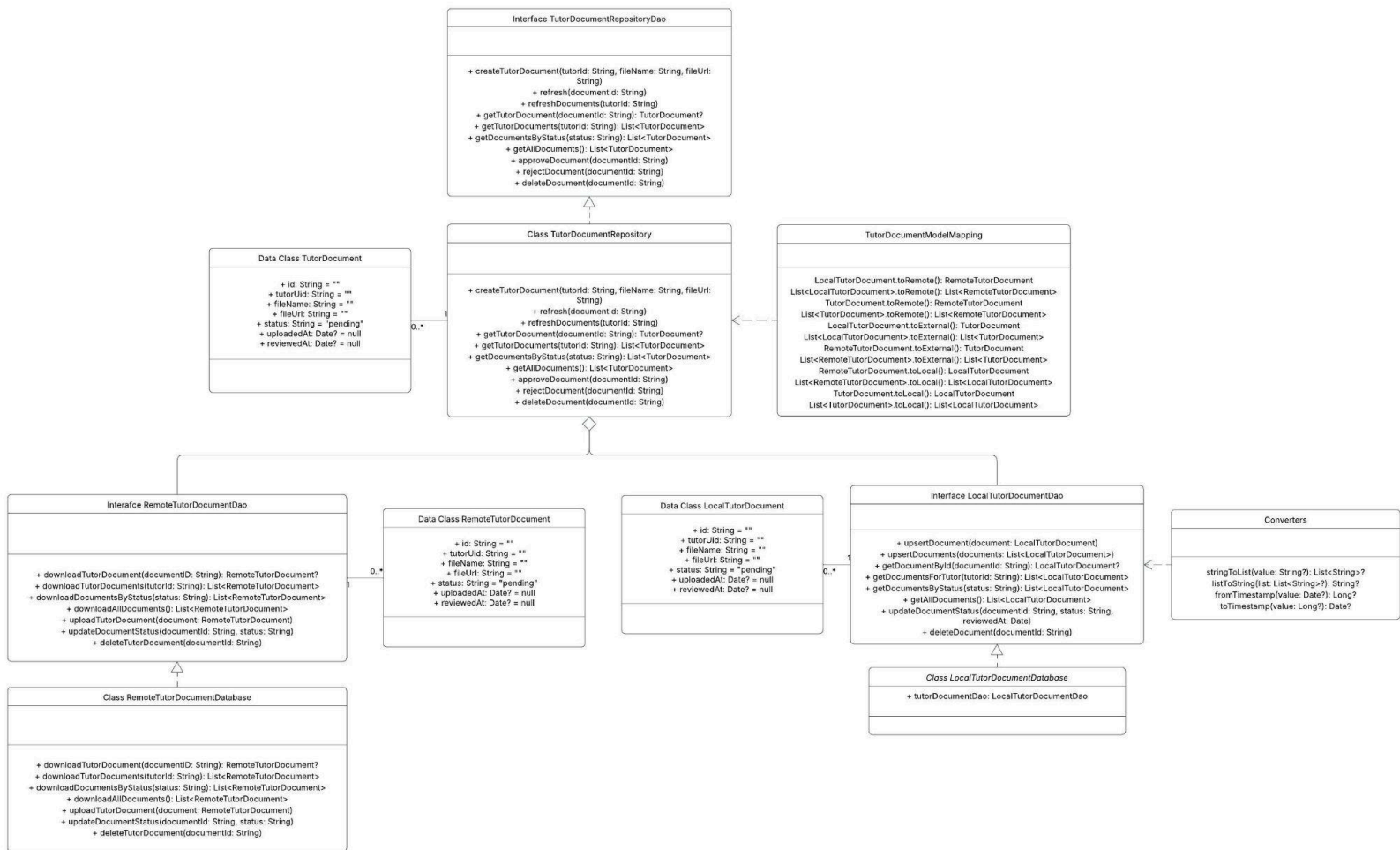
Detailed Messaging Class Diagram



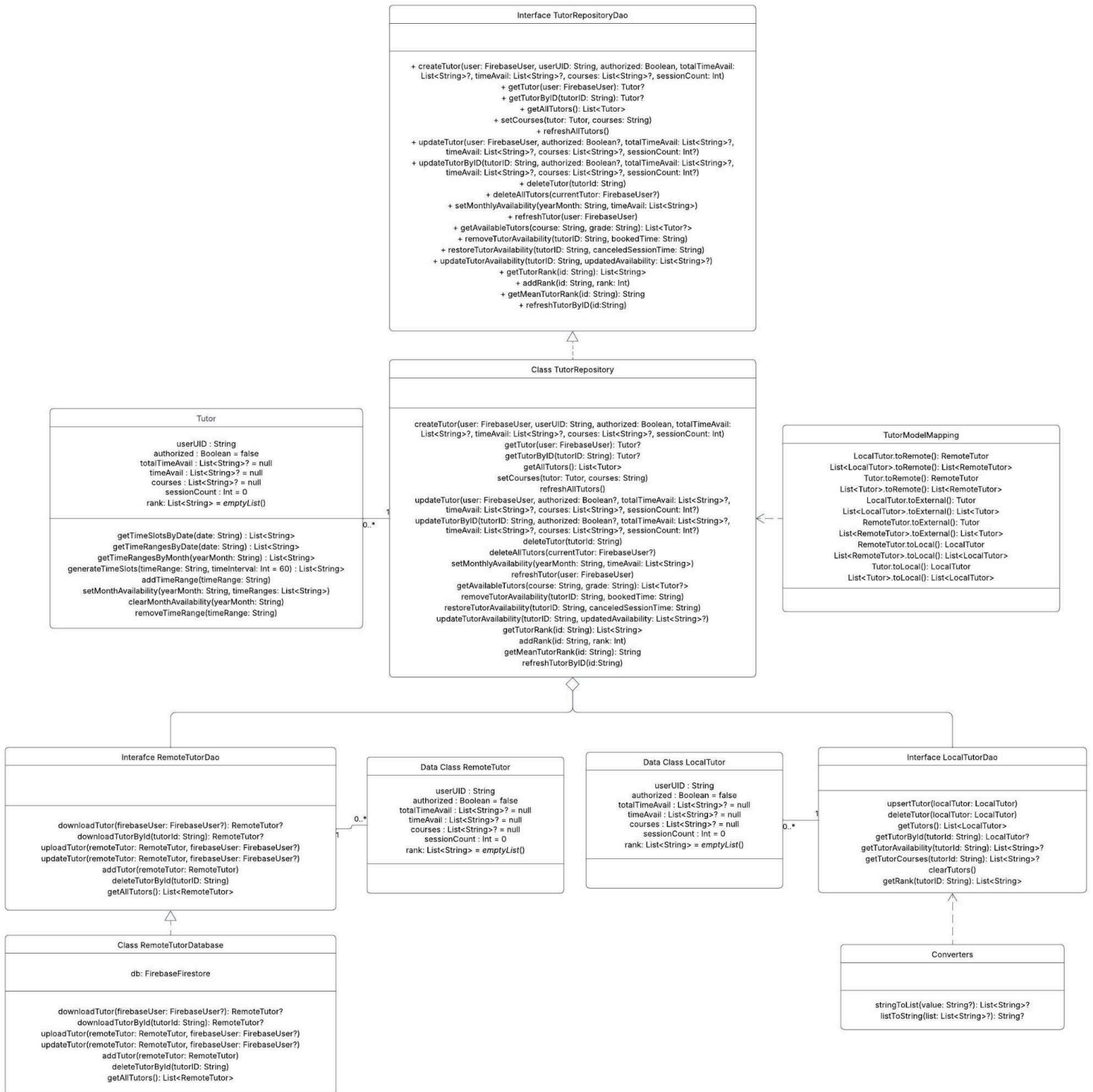
Detailed Session Database class diagram



Detailed Tutor Document Database class diagram

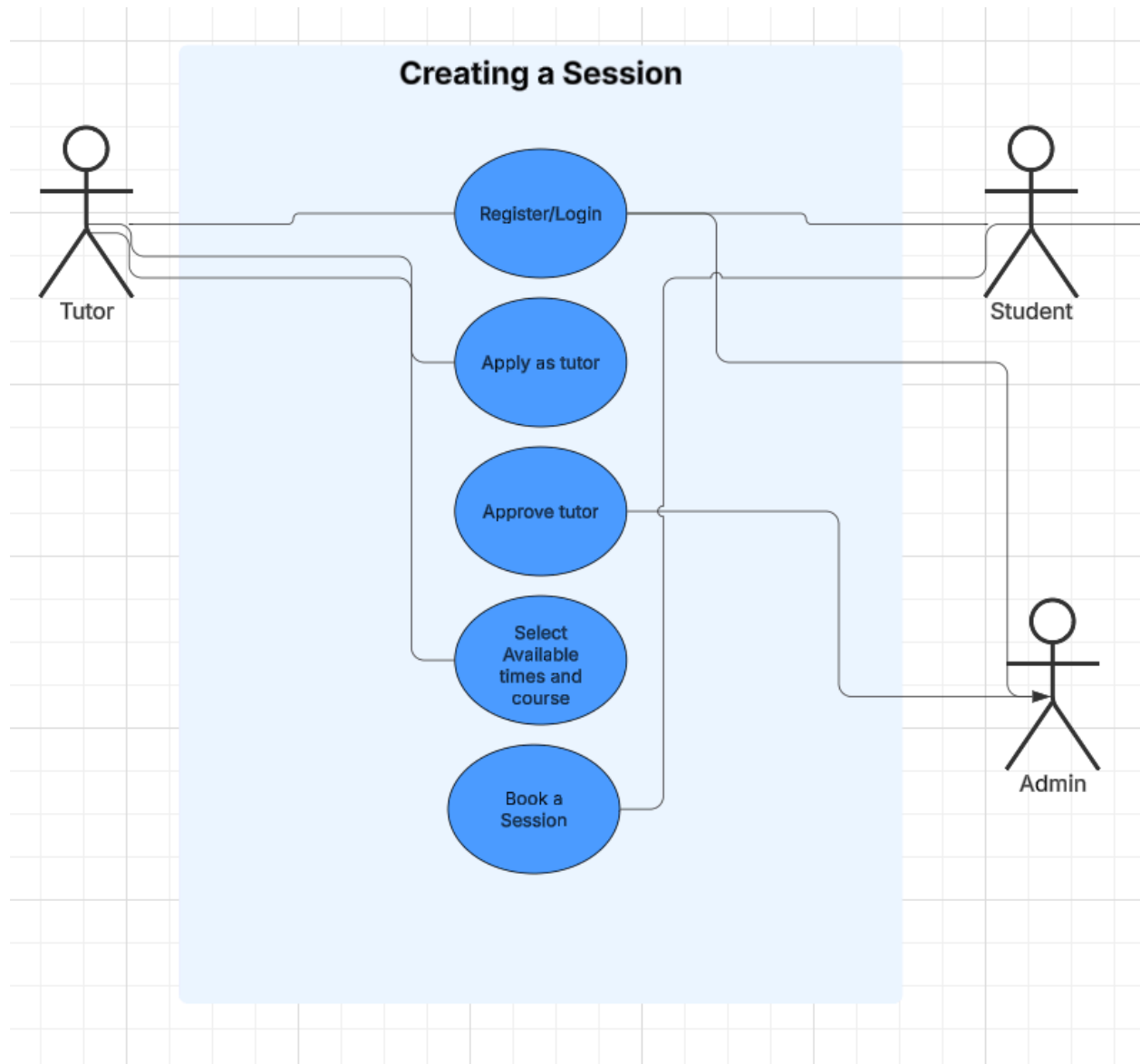


Detailed Tutor User Database class diagram

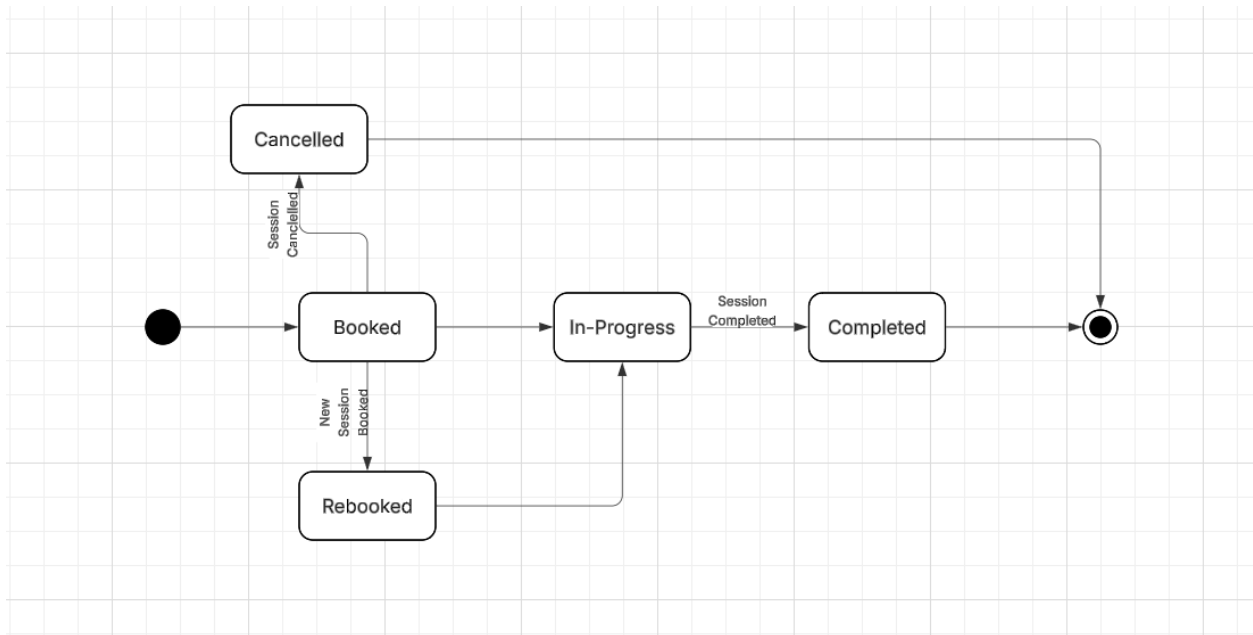


System Behaviour

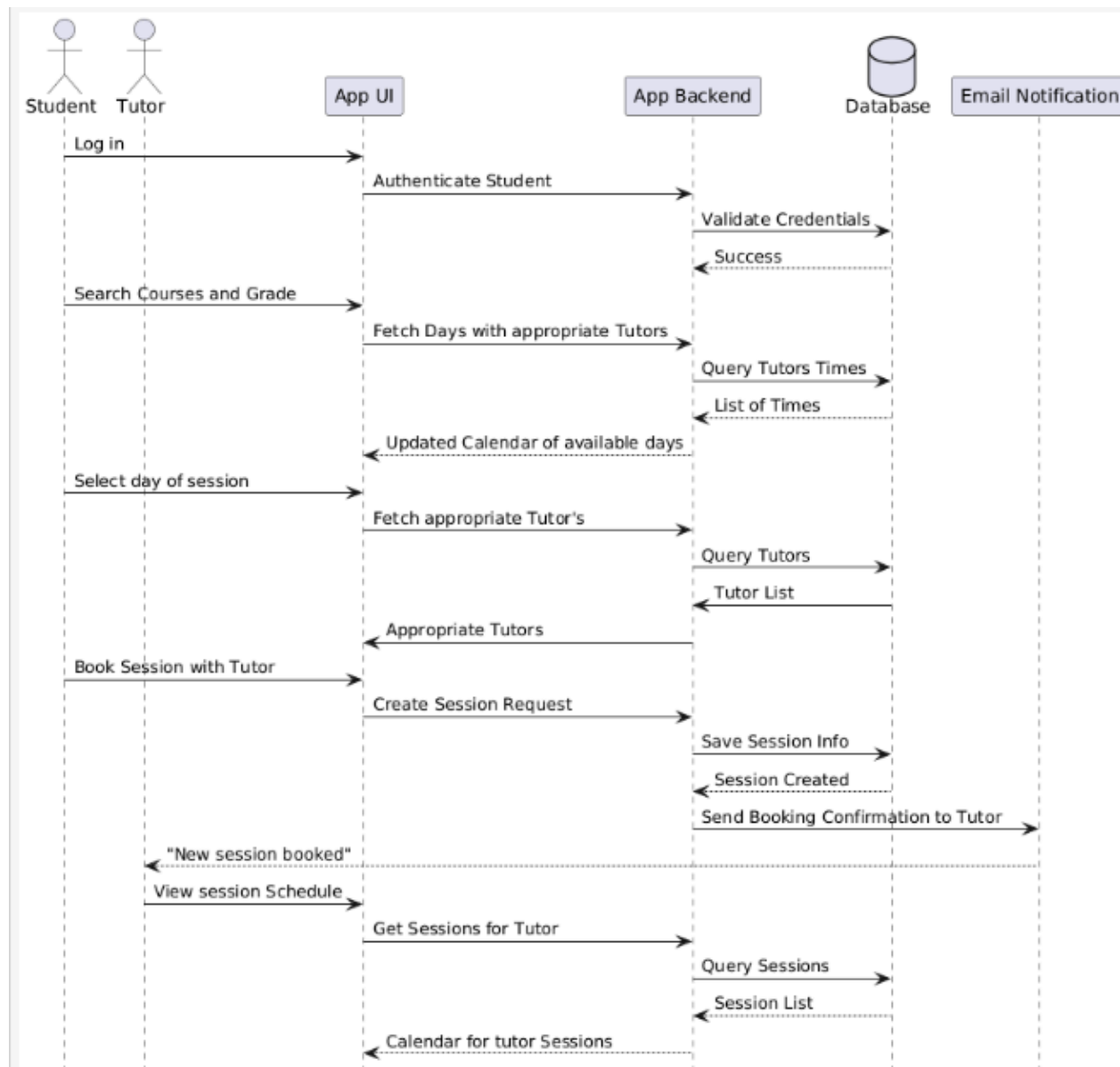
Use Case Diagram:



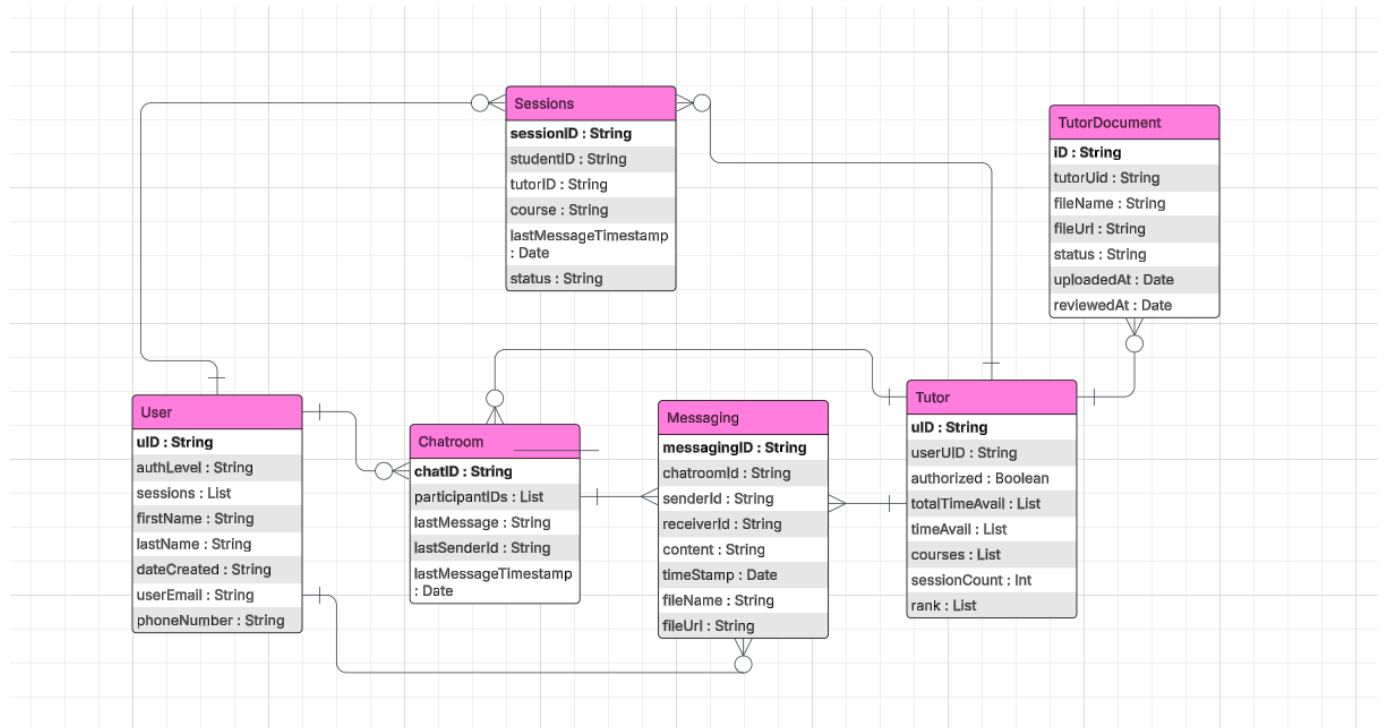
State Machine Diagram:



Sequence Diagram:



ERD



Appendix of code (Doxygen)

There was a problem generating the HTML file containing the doxygen document.

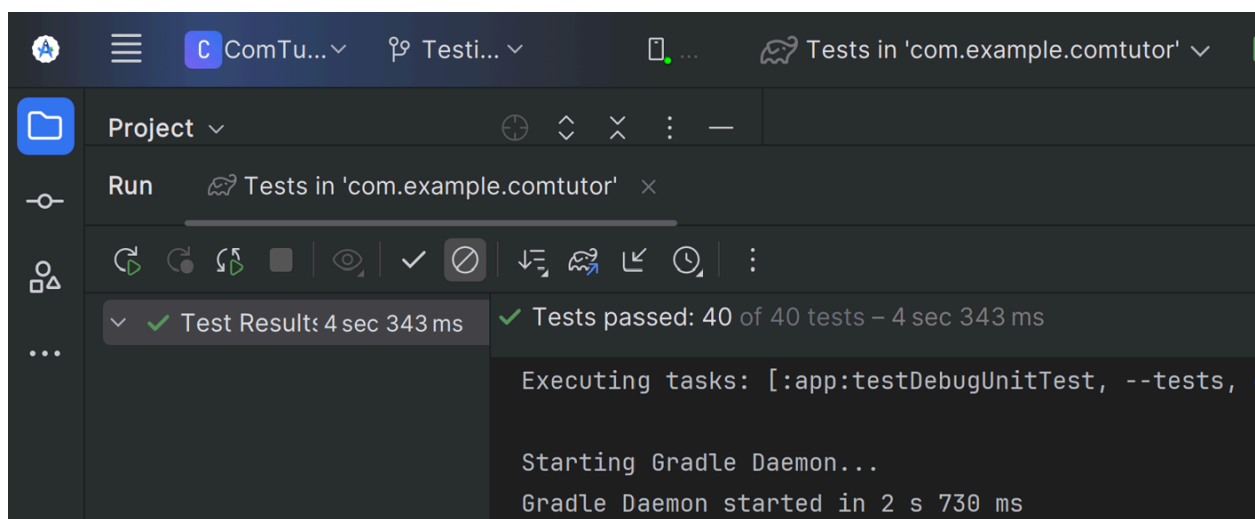
Testing Strategies

This document outlines the testing strategy and implementation for the ComTutor Android app. It includes unit testing, UI testing with Espresso, and cloud-based testing tools like Firebase Test Lab and Crashlytics. This section is intended for developers taking over the project or working on maintenance and extension.

Unit Testing Strategy

All unit tests are located in the **test/unitTest** directory. These tests are logic-based and do not require an emulator or Android device. They test ViewModels and Repositories using JUnit, Mockito, and the Kotlin coroutines test library. LiveData results are handled using a custom utility (**LiveDataTestUtil.kt**), and logs are optionally printed using **Log.kt**.

Unit test results showing all 40 tests passing.



Repository Tests

UserRepositoryTest

- **createUser uploads when user does not exist remotely:** Verifies that a new user is uploaded to Firestore and inserted into Room if they are not found remotely.
- **getUserByID returns null when remote returns null:** Ensures that null is returned gracefully when a user doesn't exist in Firestore.
- **updateUser updates names if provided:** Tests selective updating of user profile fields.
- **addSession appends session ID to list:** Confirms session tracking is working.
- **deleteUser removes user from local db:** Ensures Room database cleanup is performed.

SessionRepositoryTest

- **requestSession should return true when session is booked successfully:** Validates the end-to-end flow for successful session creation.
- **requestSession should return false if tutor is unavailable:** Confirms tutor availability is respected.
- **cancelSession should delete session and restore availability:** Ensures session cancellation correctly restores tutor availability.
- **cancelSession should return false if session not found:** Prevents crashes if cancellation is attempted on an invalid session ID.
- **getSessionByID returns session from local:** Confirms local cache is used effectively.
- **getAllSessions, getSessionsForStudent, getSessionsForTutor:** Validate session data flows using Kotlin Flows.

TutorRepositoryTest

- **createTutor stores tutor when not already existing:** Validates new tutor setup in Firestore and Room.
- **getTutor returns expected tutor after refresh:** Ensures data consistency across local and remote.
- **removeTutorAvailability updates availability list:** Tests removal of booked time slots.

- **restoreTutorAvailability adds new time slot:** Confirms correct re-adding of canceled session times.

ViewModel Tests

BookingViewModelTest

- **tutorMatching filters by course and date:** Ensures that only tutors with the correct course and available date are matched.
- **tutorMatching returns empty list when no tutor matches:** Validates negative path.
- **tutorMatching matches tutor by date only / course only:** Tests fallback behavior.
- **tutorMatching handles error from repository gracefully:** Confirms app remains stable under failure.
- **tutorMatching handles tutor with null or empty data:** Prevents null pointer crashes.

ModifyBookingViewModelTest

- **getSessionByID updates sessionDetails LiveData:** Verifies session data is displayed.
- **getSessionByID sets sessionDetails to null on error:** Ensures error handling doesn't break the UI.
- **cancelSession sets cancellationStatus to true/false:** Validates status updates.
- **cancelSession sets cancellationStatus to false on error or invalid ID:** Confirms robustness.

RegisterViewModelTest

- **createUser with student type stores user only:** Ensures students are not registered as tutors.
- **createUser with tutor type stores user and tutor:** Validates dual creation of user and tutor records.

SignInViewModelTest

- **checkIfUserExists returns true when user exists:** Confirms proper lookup.
- **checkIfUserExists returns false when user does not exist:** Ensures proper fallback.
- **checkIfUserExists returns false on exception:** Validates error handling.

TutorMatchViewModelTest

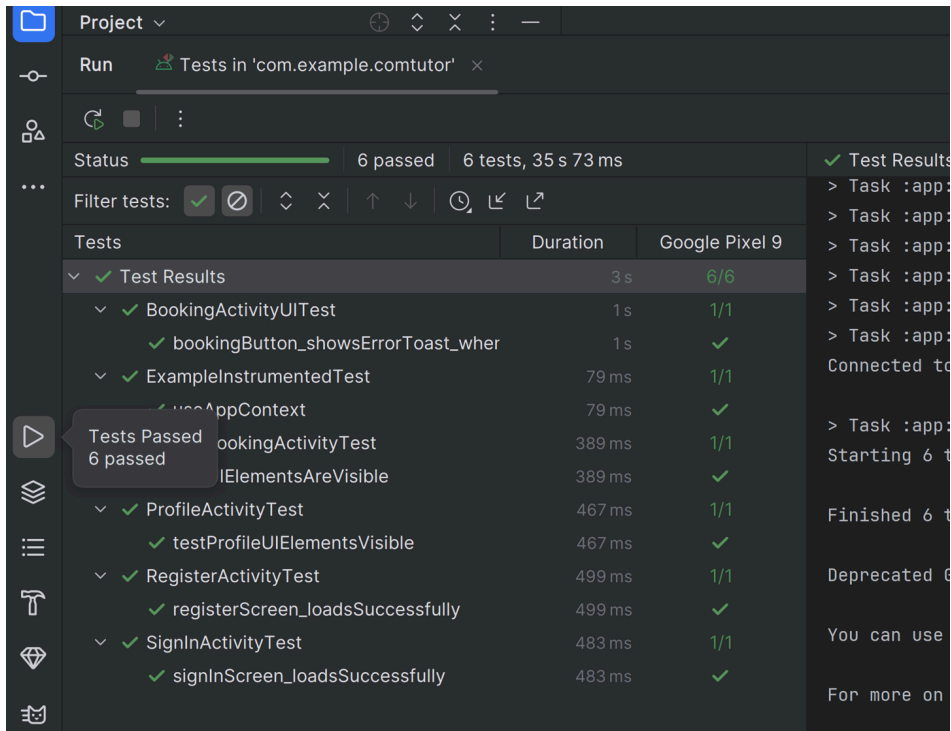
- **tutorMatching returns tutors that match course, grade, and availability:** Confirms correct filtering.
- **requestSession successfully books a session:** Validates session creation.
- **requestSession fails to book session when repository returns false:** Prevents incorrect booking feedback.

UI Testing Strategy (Espresso)

Note on Navigation Testing with Espresso: While Espresso is excellent for testing individual screens and UI components, it can be **flaky when testing navigation between multiple screens** (Activities or Fragments). This is especially true when navigation depends on asynchronous operations like Firebase calls. Espresso does not automatically wait for these background tasks, which can lead to test failures if the next screen hasn't fully loaded. For this reason, we recommend using Espresso for isolated screen tests, and manually testing full app flows on a device or emulator for the most reliable results.

UI tests are located in the **androidTest** directory. They use Espresso to test form behavior and visibility of key elements across activities.

UI test results from Espresso confirming all six activity-level tests passed.



BookingActivityUITest

- **Test:** bookingButton_showsErrorToast_whenFieldsAreNotSelected
- **What it does:** Clicks the Book button without user input
- **Why it passes:** Confirms error styling is applied to required spinners

ModifyBookingActivityTest

- **Test:** testUIElementsAreVisible
- **What it does:** Launches the screen and checks if session info, cancel button, and go-back button are present
- **Why it passes:** Ensures UI components are rendered correctly when a session ID is passed

ProfileActivityTest

- **Test:** testProfileUIElementsVisible
- **What it does:** Verifies all profile fields and navigation buttons are visible
- **Why it passes:** Confirms user profile can be viewed and navigated properly

RegisterActivityTest

- **Test:** registerScreen_loadsSuccessfully
- **What it does:** Checks that all fields and radio buttons are visible
- **Why it passes:** Ensures that the user can complete registration

SignInActivityTest

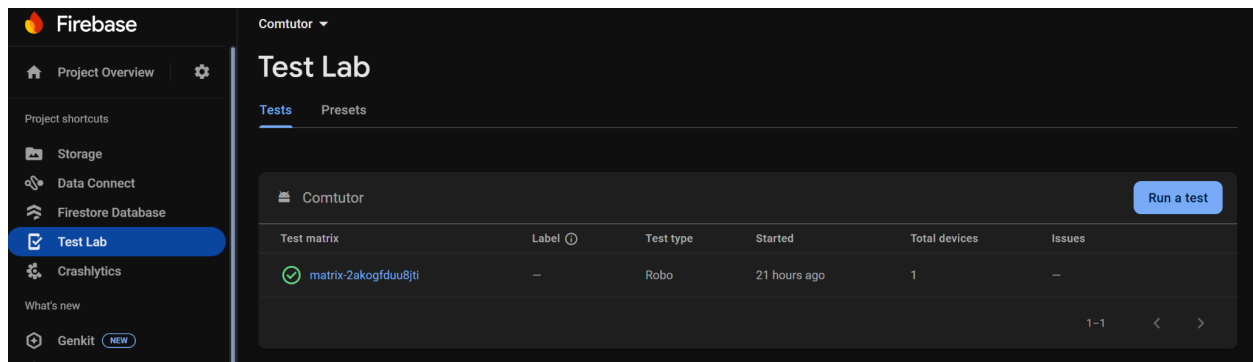
- **Test:** signInScreen_loadsSuccessfully
- **What it does:** Verifies presence of login fields, register redirect, and Google sign-in
- **Why it passes:** Confirms entry points into the app are present and functional

Firebase Test Lab & Crashlytics

Firebase Test Lab – Cloud Device Testing

Firebase Test Lab was used to run tests on a virtual **Pixel 5** device. The platform supports testing across many different hardware and Android version combinations. This provides an extra layer of validation beyond emulator testing and catches device-specific layout or logic issues.

Firebase Test Lab test run showing success on a Pixel 5 virtual device.



Firebase Crashlytics – Real-Time Crash Reporting

Crashlytics is integrated into the app to track real-world crashes and issues after deployment. While no screenshot is provided, Crashlytics allows us to:

- View detailed crash logs and stack traces
- Track crash frequency and severity
- Group similar crashes for trend monitoring

This helps developers identify and fix issues early, improving overall app stability.

Additional Notes and Recommendations

Test Coverage Summary

- This test suite covers all major ViewModels, Repository operations, and UI interactions.
- Future developers are encouraged to add tests for edge cases, performance scenarios, and full user flows as the app grows.

Known Limitations

- Navigation between activities is tested manually due to limitations with Espresso and asynchronous workflows.
- Google Sign-In and third-party Intent flows are not covered by automated tests and should be validated manually during QA.
- Crashlytics reporting is integrated but was not stress-tested with simulated crash cases.

Summary

- Unit tests cover Repositories and ViewModels using JUnit and Mockito
- UI tests validate screen-level behavior using Espresso
- Firebase Test Lab and Crashlytics provide cloud-based support for cross-device testing and runtime monitoring

This testing strategy provides a comprehensive safety net and can be extended by future developers as the app evolves.

Bug Tracking

This section outlines bugs, what caused them, the type and severity of the bug, who tracked them, and if it was resolved.

1) Cancel session not working, it doesn't add the time available back.

- **Type:** Algorithmic, Critical
- **Tracked by:** Jim
- **Root Cause:** Misplaced logic, finished function before call was made to the method to add the time back.
- **Bug State:** Solved

2) The entirety of the session list is shown on the modify booking page, even when it's only supposed to show the sessions associated with the user.

- **Type:** Algorithmic, Critical
- **Tracked by:** Zhire/Dahlin
- **Root Cause:** Management of on device storage not working properly, sessions are completely downloaded and then not deleted upon finishing task.
- **Bug State:** Solved

3) A toast message saying "Google sign-in failed" even when the user can log in.

- **Type:** Algorithmic, Non Critical
- **Tracked by:** Team
- **Root Cause:** The toast warning is put in the wrong section of the code.
- **Bug State:** Not Solved

4) The dates on the calendar view with sessions scheduled in them are not highlighted.

- **Type:** Algorithmic, Non Critical
- **Tracked by:** Zhire
- **Root Cause:** The logic is put in the wrong section of the code.
- **Bug State:** Solved

5) The format of the chat window is wrong. The distance between each text bubble is the length of the chat window.

- **Type:** Algorithmic, Non Critical

- **Tracked by:** James
- **Root Cause:** The XML files for the chat bubbles used two layouts instead of just one, which caused the distance to be bigger than it should have been.
- **Bug State:** Solved

6) The text box for the last message sent has no height limit. It will take up the whole screen if the message is long enough.

- **Type:** Algorithmic, Non Critical
- **Tracked by:** Dahlin
- **Root Cause:** The height is set to incrementally get bigger the more text is in the textbox.
- **Bug State:** Not Solved

7) The list view in the schedule page only shows the one scheduled session. If there are multiple sessions scheduled on the same day, it only shows the last entry.

- **Type:** Algorithmic, Non Critical
- **Tracked by:** Zhire
- **Root Cause:** TextView is used to display the information instead of ListView.
- **Bug State:** Solved

8) Modifying a session was not working properly. It only cancels the existing session and doesn't rebook a new one.

- **Type:** Algorithmic, Critical
- **Tracked by:** Zhire
- **Root Cause:** The logic is put in the wrong section of the code.
- **Bug State:** Solved

9) Selecting a 2 hour block causes the booking to not be handled properly.

- **Type:** Algorithmic, Critical
- **Tracked by:** James
- **Root Cause:** Improper logic when selecting multiple time slots at once.

- **Bug State:** Not Solved

10) Downloading user created after session implementation causes app to crash.

- **Type:** Algorithmic, Critical
- **Tracked by:** Dahlin
- **Root Cause:** Constructor of class had no default value, causing one value in the object to be read without being created.
- **Bug State:** Solved